

Week 2 Fraud Detection

Project: Customer/Data Science Internship

Path: C:\workspace\Decodelabs-Data-Science-Internship\Week_2_Fraud_Detection

Week 2 — Fraud Detection Report

Overview

This report summarizes the Week 2 fraud detection pipeline: data, preprocessing, modeling, evaluation, and produced artifacts. The pipeline was run on a representative subsample by default to ensure timely development and reproducible results.

Data

- Source file used: ``Week_2_Fraud_Detection/data/creditcard.csv`` (replaced with user-provided ``transactions_train.csv``). - Rows processed (original): 6,351,193 (full dataset). Default subsample used for training: 300,000 rows (stratified by ``isFraud``).

Preprocessing & Feature Engineering

- Basic cleaning: fill/mask missing values, drop unexpected columns where necessary. - New features: ``orig_balance_delta``, ``dest_balance_delta``, one-hot encoding of ``type``, and other domain-relevant transforms in ``src/preprocessing.py``. - Scaling: numeric features scaled with ``StandardScaler`` where appropriate. - Resampling: ``SMOTE`` applied to the training set to address class imbalance when training classifiers.

Models Trained

- Logistic Regression (``class_weight='balanced'``, ``max_iter=1000``). - Random Forest (``n_estimators=200``, ``class_weight='balanced'``). - Gradient Boosting (``n_estimators=200``). Models were trained on the subsample by default; model artifacts are saved to ``Week_2_Fraud_Detection/models/``.

Evaluation & Results

- Evaluation artifacts (ROC curves, precision-recall, confusion matrices, feature importances) saved in ``Week_2_Fraud_Detection/charts/``. - Predictions for the test split saved to ``Week_2_Fraud_Detection/outputs/fraud_predictions.csv``. - Summary: classifiers produced separable ROC/PR curves on the subsample; consult the plots in ``charts/`` for metric details and thresholds.

Reproducibility

To reproduce the default pipeline (subsample run): ````bash python Week_2_Fraud_Detection/run_week2.py ```` To run on the full dataset (resource-heavy), re-run with ``max_rows=None`` or modify the script's sampling parameter — be prepared for long runtime and high memory usage.

Artifacts

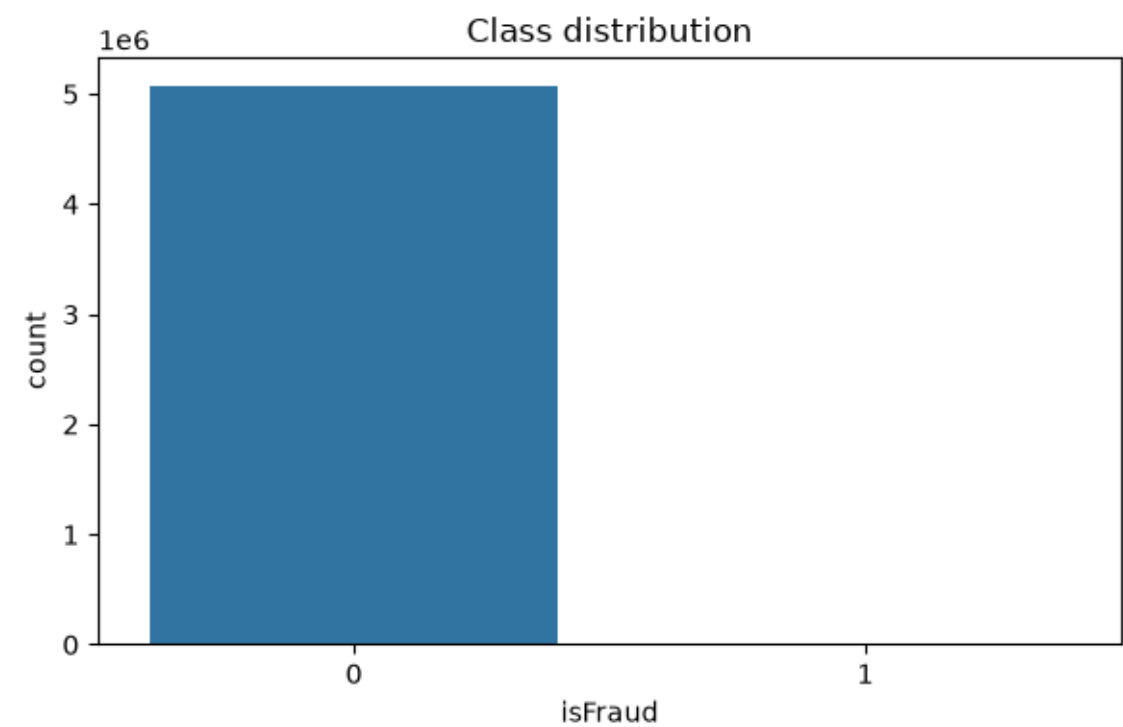
- Models: ``Week_2_Fraud_Detection/models/random_forest.pkl``, ``gradient_boosting.pkl``, ``logistic_regression.pkl``. - Charts: ``Week_2_Fraud_Detection/charts/`` (ROC, PR, confusion matrices, feature importance). - Predictions: ``Week_2_Fraud_Detection/outputs/fraud_predictions.csv``. - Notebooks: ``Week_2_Fraud_Detection/notebooks/fraud_detection.ipynb`` and ``fraud_detection_executed.ipynb`` (executed summary).

Next Steps & Recommendations

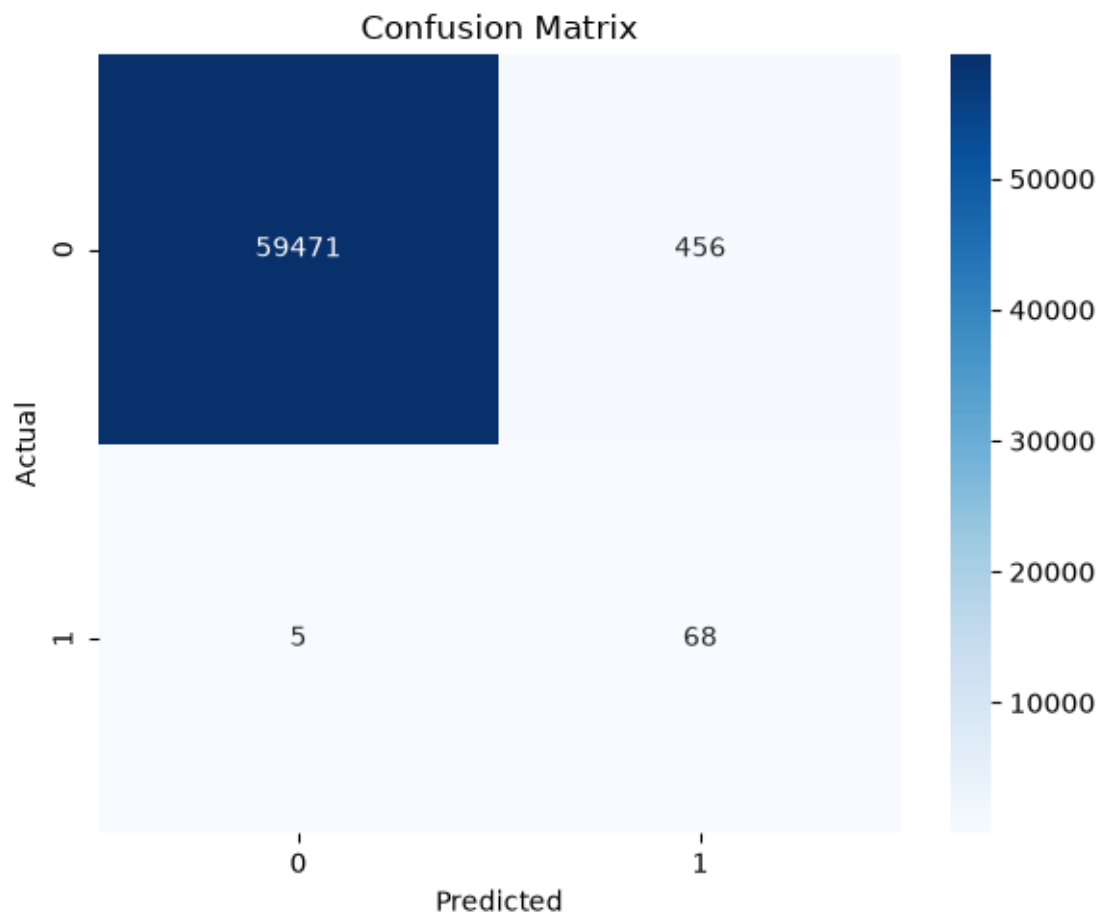
- Optionally implement and validate an XGBoost model in ``src/model_training.py`` (I can implement this and run on the subsample). - If final portfolio requires full-dataset models, schedule a dedicated run on a machine with sufficient RAM/CPU or use incremental/online training. - Draft a concise ``Week2_Report.md`` (this file) and then finalize formatting/figures for portfolio presentation. ---- Generated: 2026-06-15

KPI	Value
Dataset size	N/A
Features (sample)	N/A
Missing values	N/A
Models trained	3

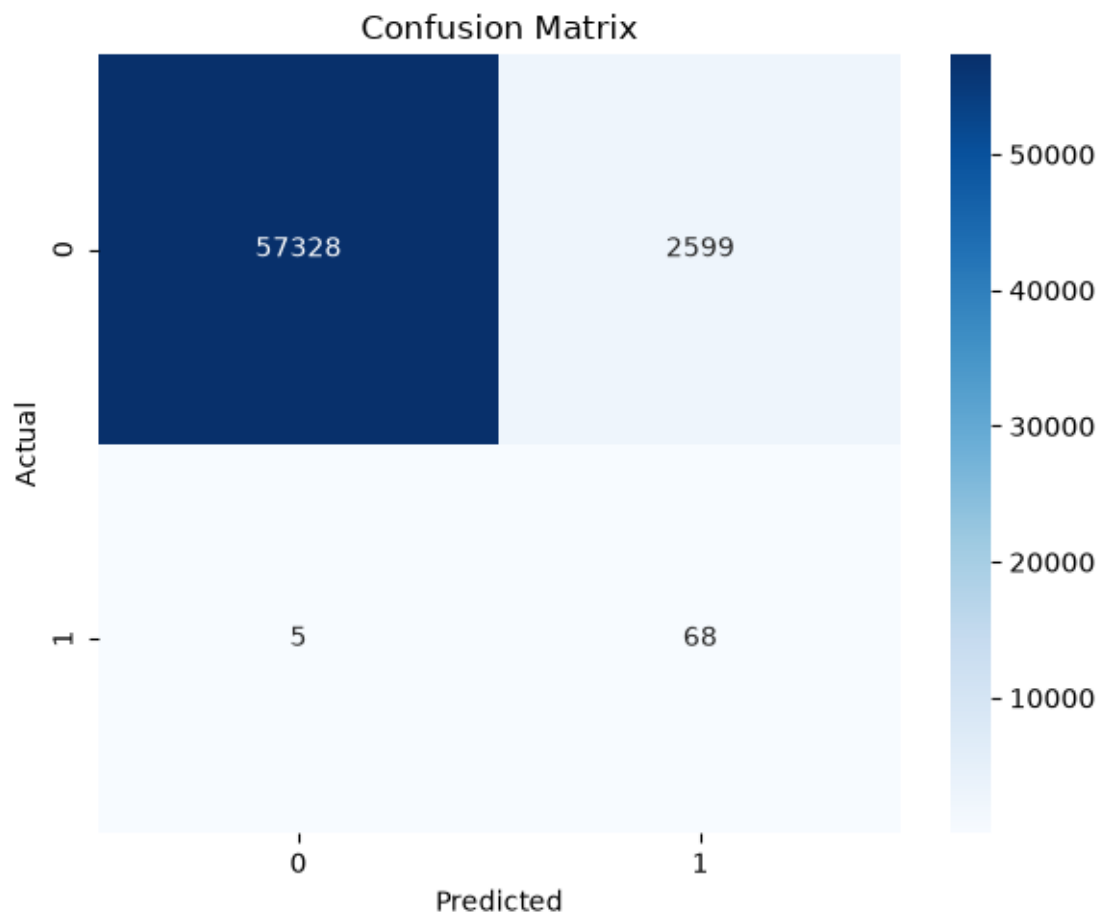
class distribution



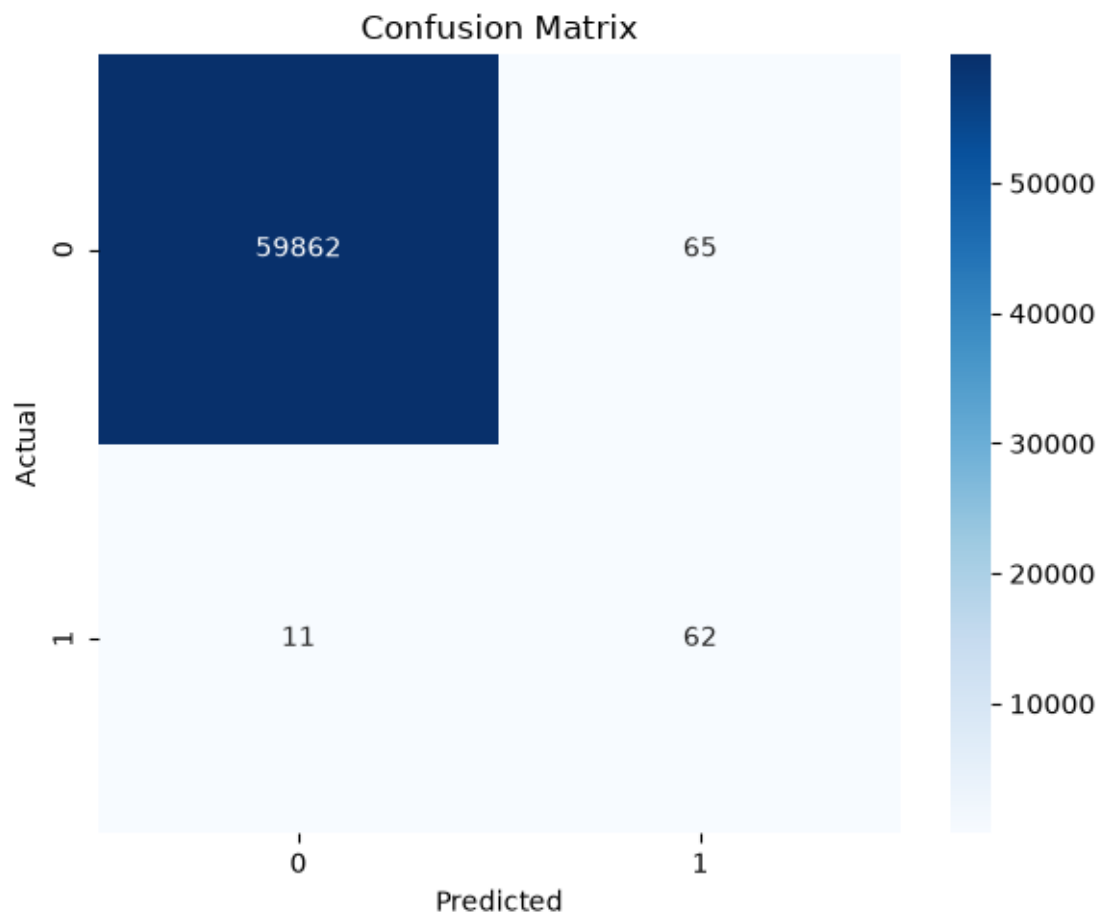
confusion matrix gradient boosting



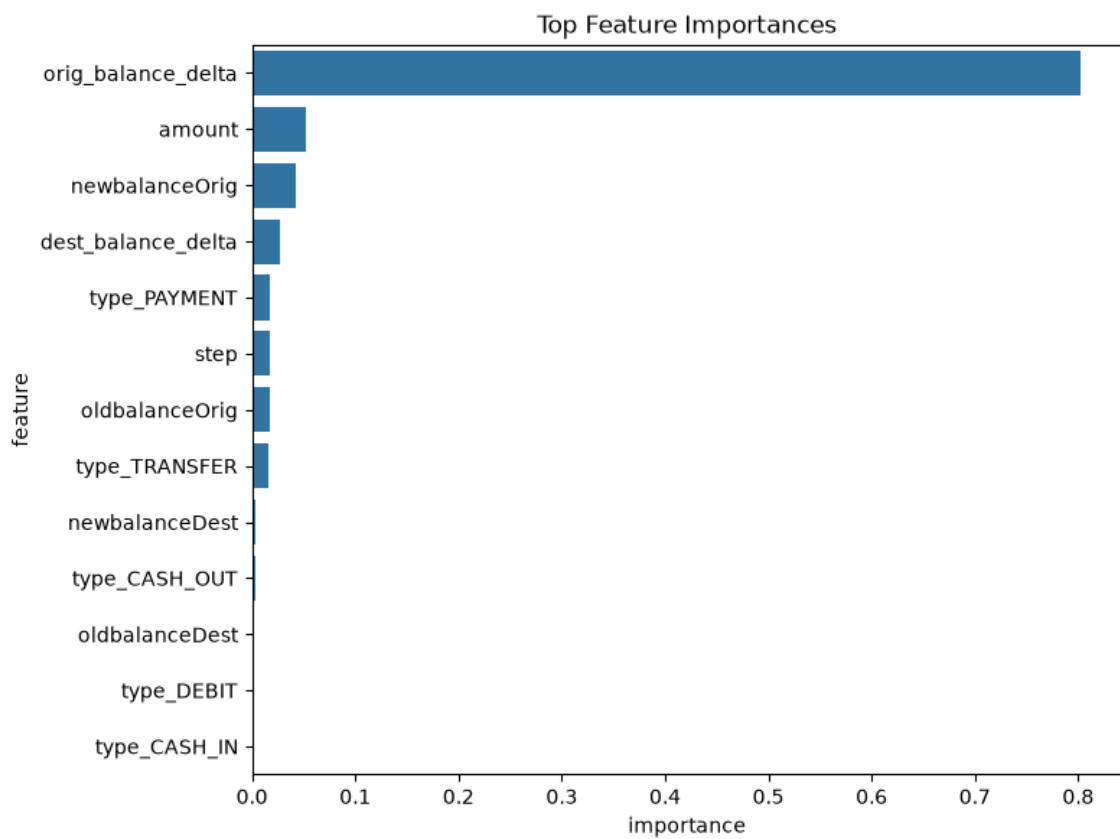
confusion matrix logistic regression



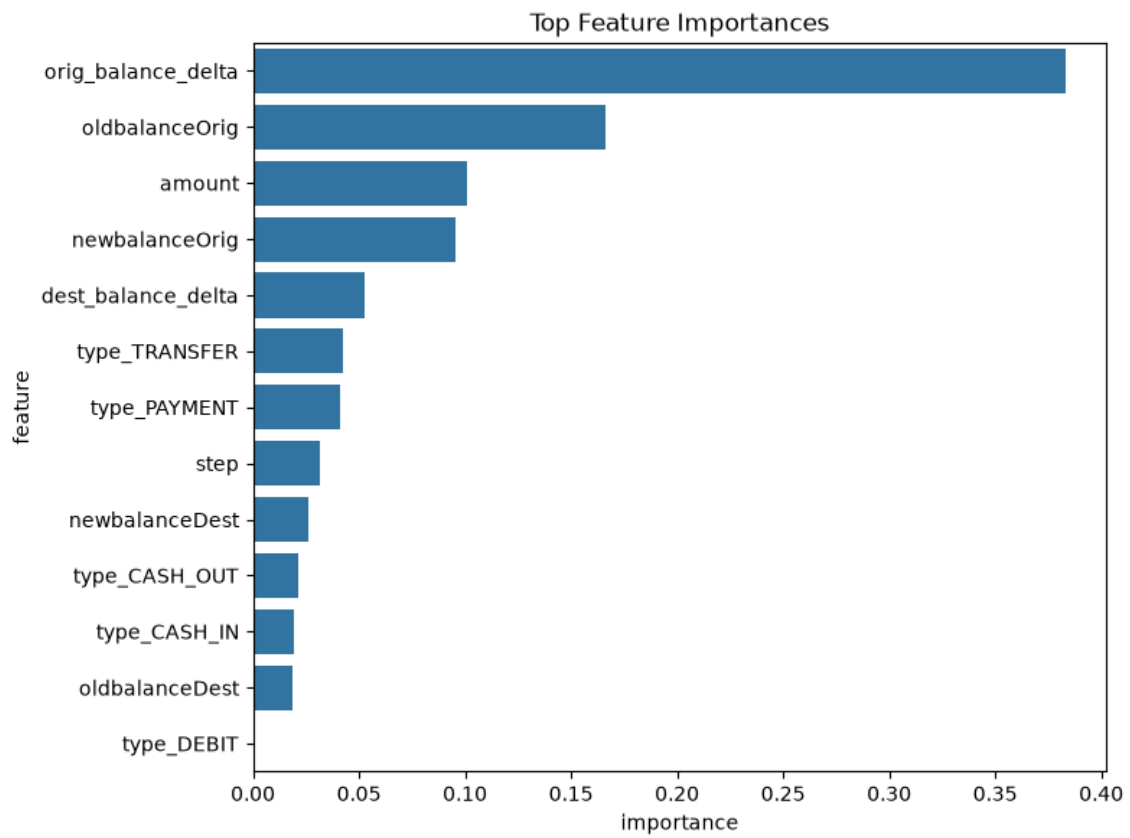
confusion matrix random forest



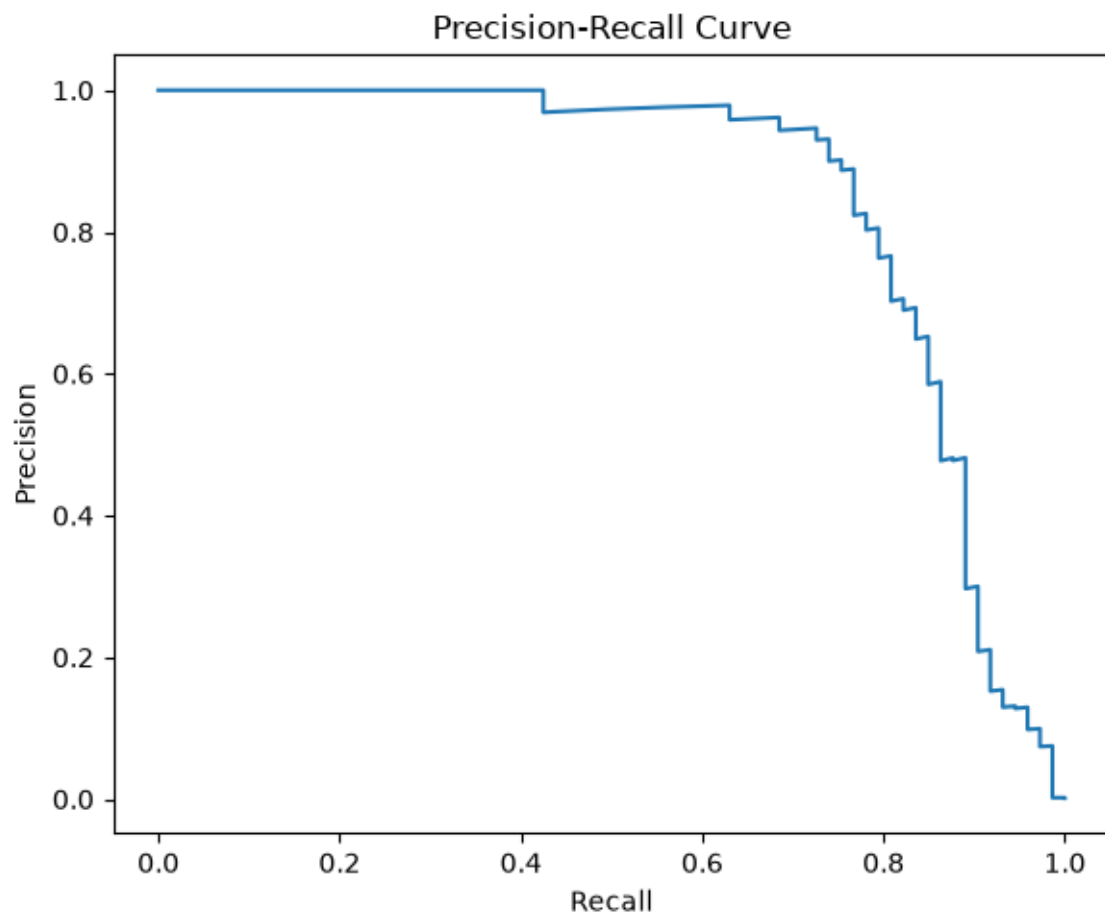
feature importance gradient boosting



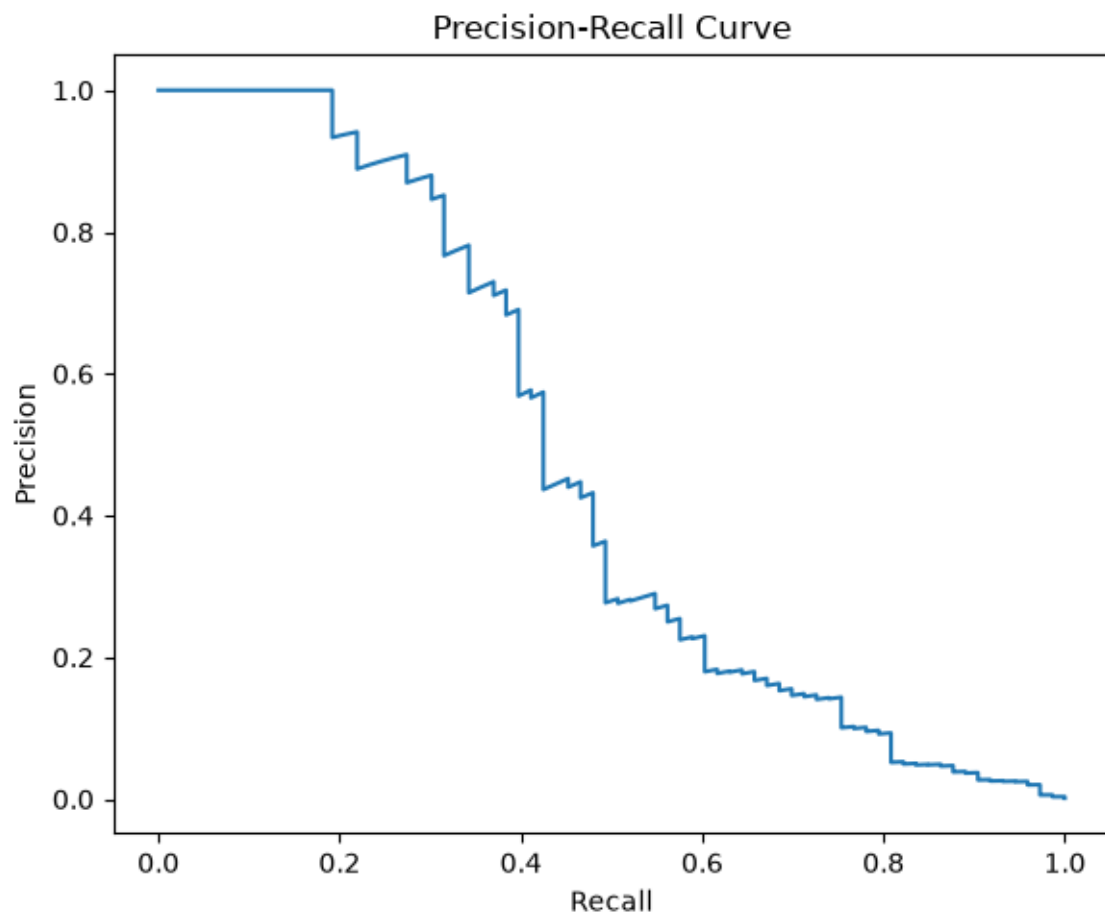
feature importance random forest



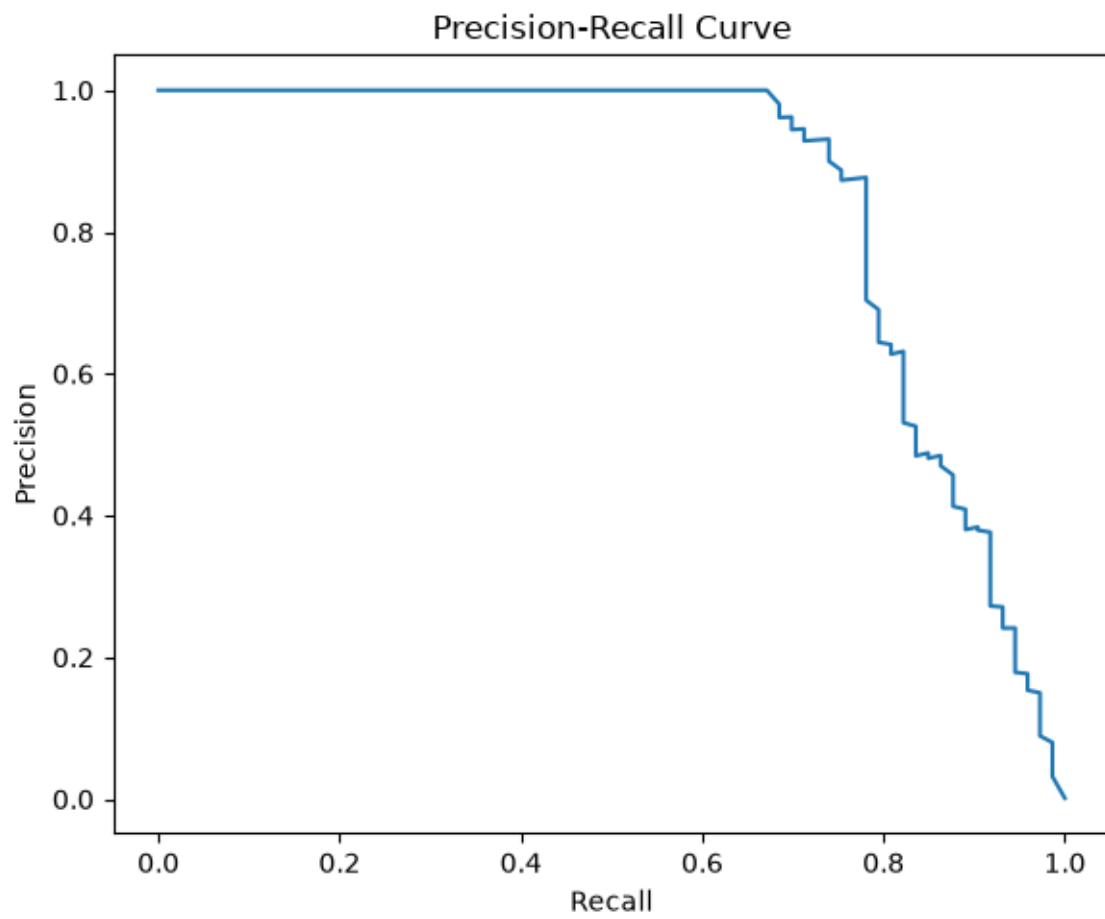
precision recall gradient boosting



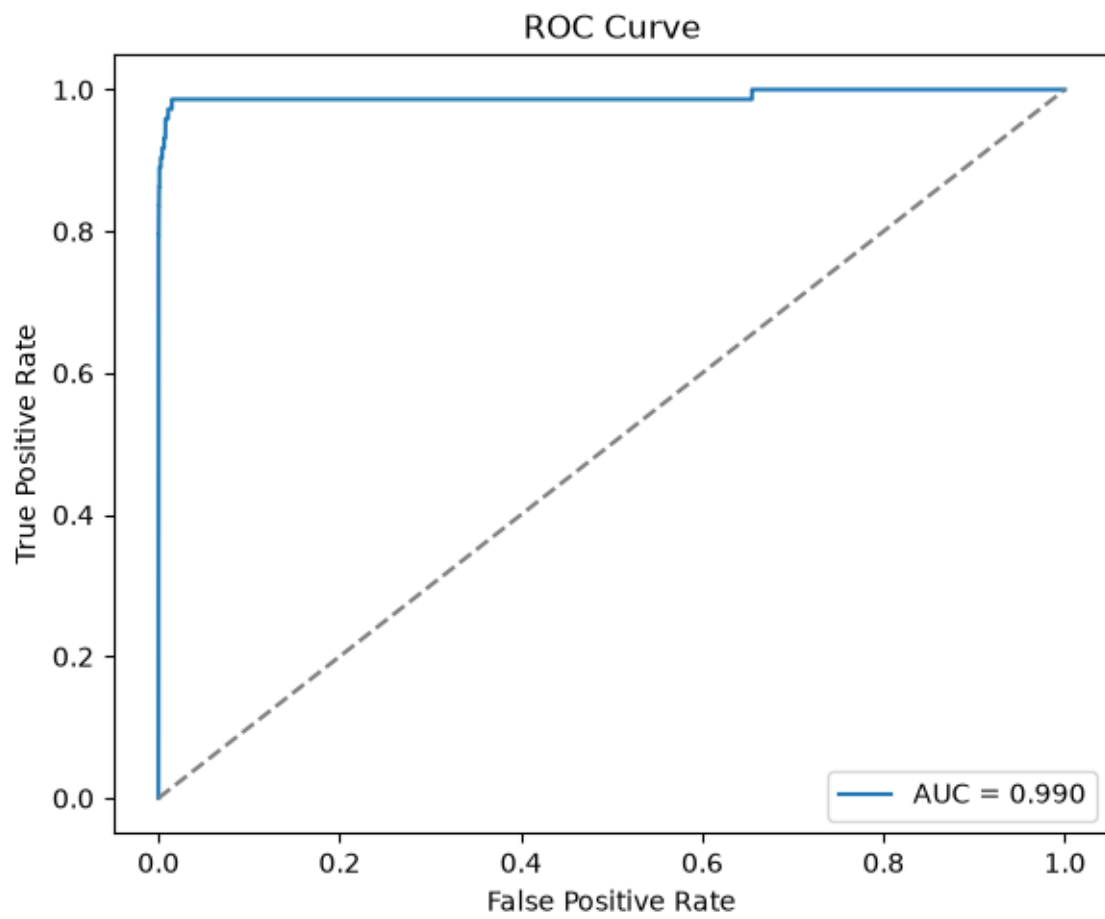
precision recall logistic regression



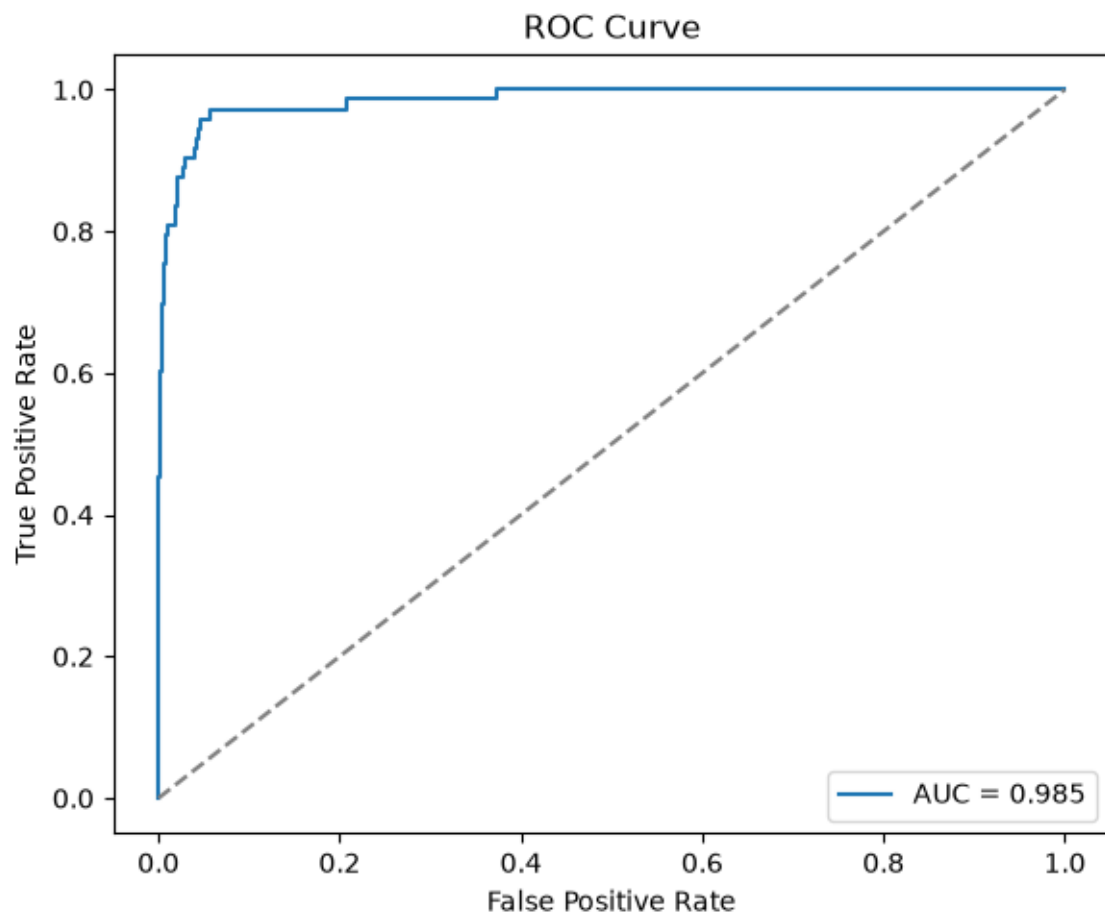
precision recall random forest



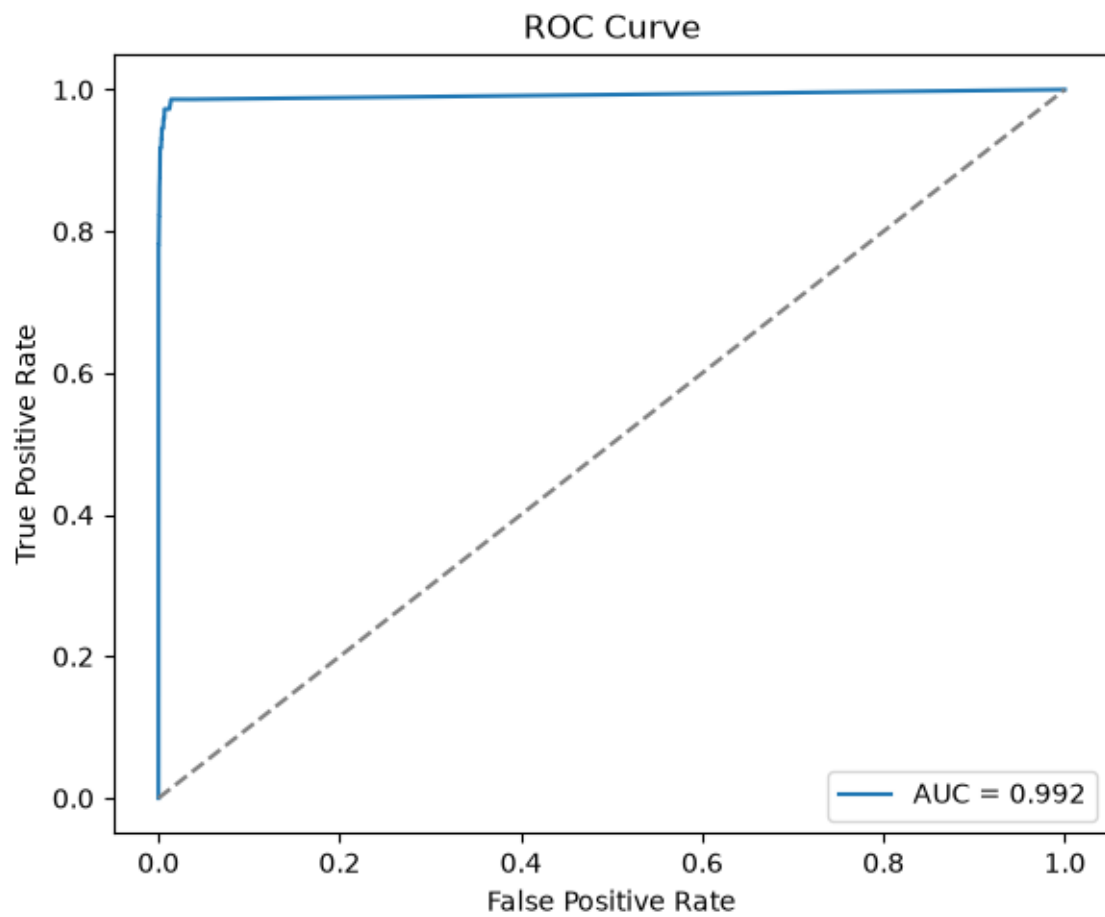
roc curve gradient boosting



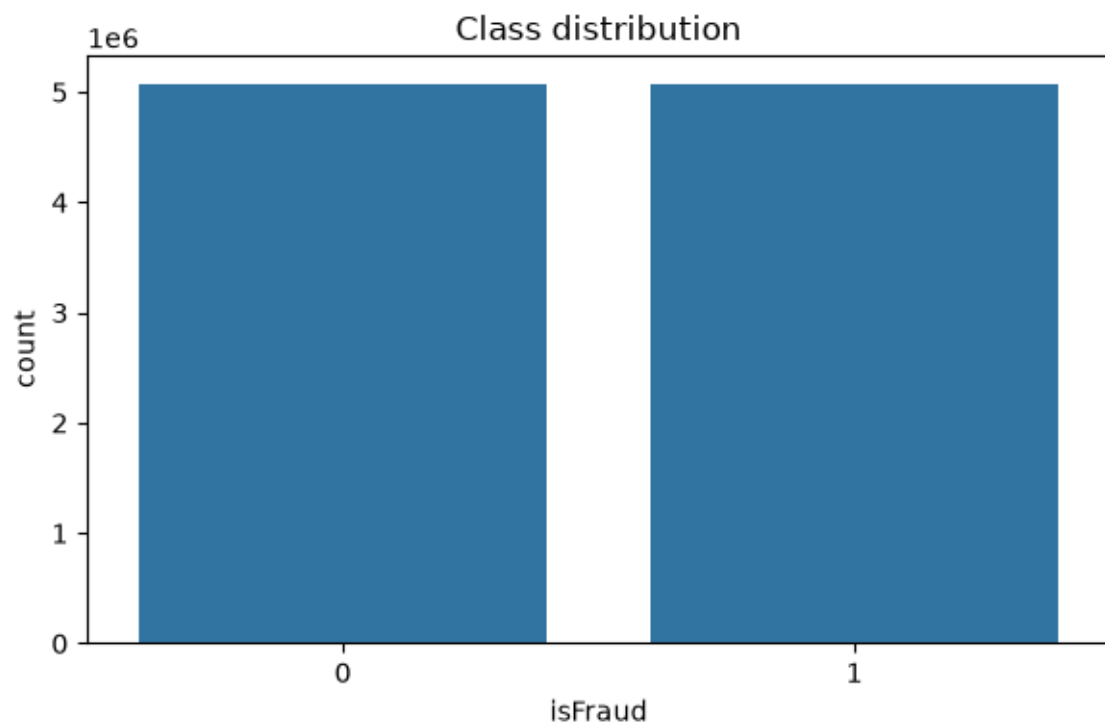
roc curve logistic regression



roc curve random forest



smote distribution



Appendix

Week 2 — Fraud Detection Report

Overview

This report summarizes the Week 2 fraud detection pipeline: data, preprocessing, modeling, evaluation, and produced artifacts. The pipeline was run on a representative subsample by default to ensure timely development and reproducible results.

Data

- Source file used: `Week\_2\_Fraud\_Detection/data/creditcard.csv` (replaced with user-provided `transactions\_train.csv`).
- Rows processed (original): 6,351,193 (full dataset). Default subsample used for training: 300,000 rows (stratified by `isFraud`).

Preprocessing & Feature Engineering

- Basic cleaning: fill/mask missing values, drop unexpected columns where necessary.
- New features: `orig\_balance\_delta`, `dest\_balance\_delta`, one-hot encoding of `type`, and other domain-relevant transforms in `src/preprocessing.py`.
- Scaling: numeric features scaled with `StandardScaler` where appropriate.
- Resampling: `SMOTE` applied to the training set to address class imbalance when training classifiers.

Models Trained

- Logistic Regression (`class\_weight='balanced'`, `max\_iter=1000`).
- Random Forest (`n\_estimators=200`, `class\_weight='balanced'`).
- Gradient Boosting (`n\_estimators=200`).

Models were trained on the subsample by default; model artifacts are saved to `Week\_2\_Fraud\_Detection/models/`.

Evaluation & Results

- Evaluation artifacts (ROC curves, precision-recall, confusion matrices, feature importances) saved in `Week\_2\_Fraud\_Detection/charts/`.
- Predictions for the test split saved to `Week\_2\_Fraud\_Detection/outputs/fraud\_predictions.csv`.
- Summary: classifiers produced separable ROC/PR curves on the subsample; consult the plots in `charts/` for metric details and thresholds.

Reproducibility

To reproduce the default pipeline (subsample run):

```
```bash
python Week_2_Fraud_Detection/run_week2.py
```
```

To run on the full dataset (resource-heavy), re-run with `max\_rows=None` or modify the script's sampling parameter — be prepared for long runtime and high memory usage.

Artifacts

- Models: `Week\_2\_Fraud\_Detection/models/random\_forest.pkl`, `gradient\_boosting.pkl`, `logistic\_regression.pkl`.
- Charts: `Week\_2\_Fraud\_Detection/charts/` (ROC, PR, confusion matrices, feature importance).
- Predictions: `Week\_2\_Fraud\_Detection/outputs/fraud\_predictions.csv`.
- Notebooks: `Week\_2\_Fraud\_Detection/notebooks/fraud\_detection.ipynb` and `fraud\_detection\_executed.ipynb` (executed summary).

Next Steps & Recommendations

- Optionally implement and validate an XGBoost model in `src/model\_training.py` (I can implement this and run on the subsample).
- If final portfolio requires full-dataset models, schedule a dedicated run on a machine with sufficient RAM/CPU or use incremental/online training.
- Draft a concise `Week2\_Report.md` (this file) and then finalize formatting/figures for portfolio presentation.

Generated: 2026-06-15